

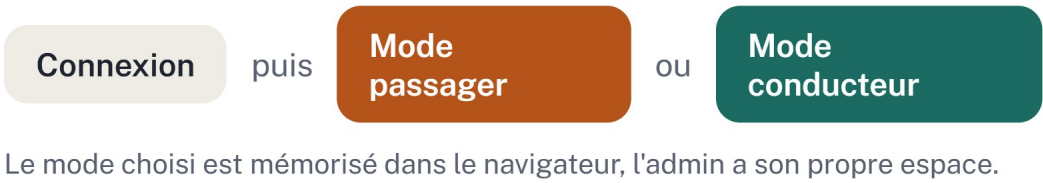
Roulons, l'application de covoiturage

Des conducteurs publient leurs trajets. Des passagers les trouvent sur une carte, réservent une ou plusieurs places et discutent avec le conducteur avant le départ. Le projet se juge sur la solidité de l'ingénierie : jamais une place vendue deux fois, des données protégées, une API testée et documentée.

Un compte, deux modes

On ne choisit pas une fois pour toutes « conducteur » ou « passager » à l'inscription. Chaque utilisateur choisit son mode à la connexion et peut basculer à tout moment depuis le bandeau du haut. La même personne peut proposer un trajet le vendredi et en réserver un le dimanche.

Côté back, il n'y a donc que deux rôles de sécurité : **USER** et **ADMIN**. Le mode ne change que l'interface affichée.



Passager

- Cherche un trajet par villes, date et nombre de places
- Voit les trajets sur la carte
- Réserve une ou plusieurs places
- Annule, consulte son historique
- Discute avec le conducteur

Conducteur

- Publie un trajet : villes, horaire, places, prix
- Modifie ou annule ses trajets
- Voit qui a réservé et combien de places restent
- Discute avec ses passagers

Admin

- Traite les signalements des utilisateurs
- Masque un trajet ou supprime un message
- Suspend un compte

Les règles métier, celles que le jury va tester

- 1

Jamais plus de réservations que de places, même si deux passagers cliquent à la même seconde. Le décompte des places se fait dans la même opération que la réservation.
- 2

On ne réserve pas n'importe quoi : ni son propre trajet, ni un trajet complet, annulé ou déjà parti.
- 3

Le conducteur ne peut pas retirer des places déjà prises : le nombre de places ne descend jamais sous le nombre réservé.
- 4

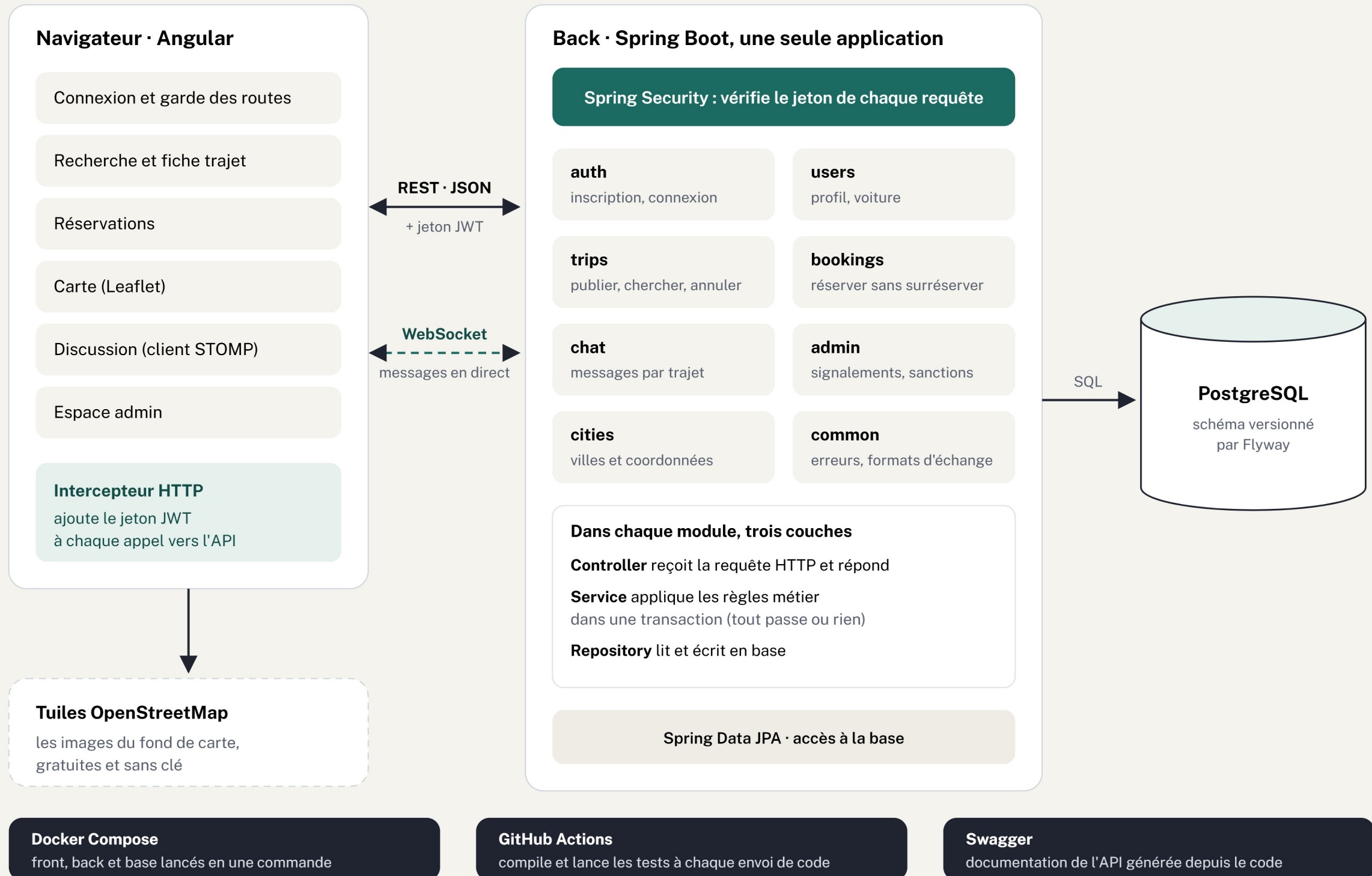
Chacun ne touche qu'à ses données. Le serveur vérifie toujours le propriétaire, même si l'écran cache déjà le bouton.
- 5

La discussion d'un trajet est privée : seuls le conducteur et les passagers confirmés peuvent la lire et y écrire.
- 6

Annuler un trajet annule toutes ses réservations, en une seule fois, et les passagers le voient dans leur historique.

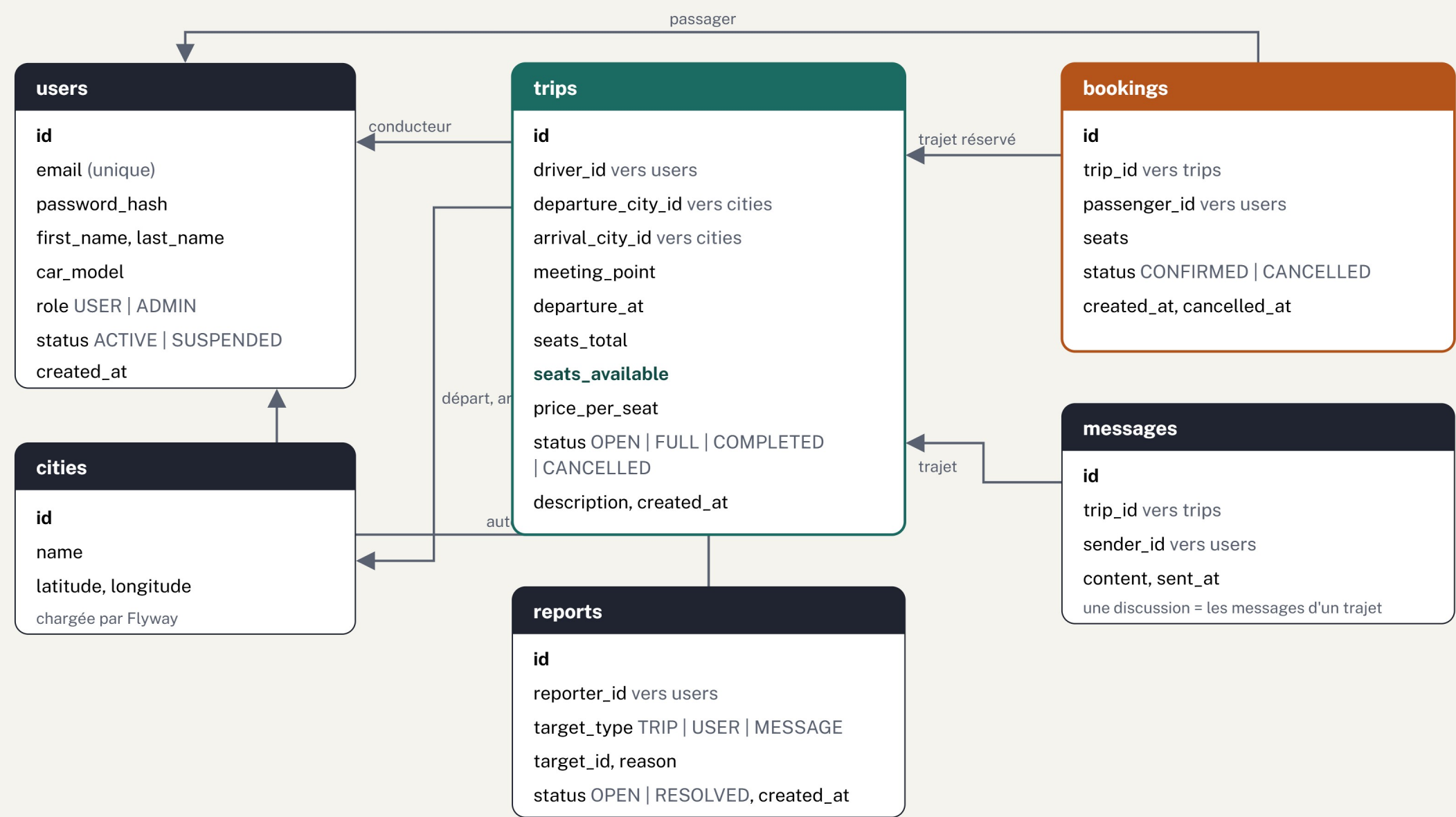
Architecture

Une seule application Spring Boot découpée en modules, un front Angular et une base PostgreSQL. Deux canaux entre le navigateur et le serveur : des appels REST classiques, et une connexion WebSocket qui reste ouverte pour que les messages du chat arrivent sans recharger la page.



Modèle de données

Six tables. Une flèche va de la table qui contient la référence vers la table référencée. Les valeurs en majuscules sont les statuts possibles.



Réserver sans jamais surréserver

On retire les places avec une seule requête qui vérifie en même temps qu'il en reste assez :

```
UPDATE trips
SET seats_available = seats_available - :n
WHERE id = :id AND status = 'OPEN'
AND seats_available >= :n
```

Si aucune ligne n'est modifiée, il ne restait pas assez de places : on refuse. La création de la réservation est dans la même transaction.

La base protège aussi

Même si le code avait un bug, des contraintes posées dans les scripts Flyway empêchent les données incohérentes :

- places restantes toujours entre 0 et le total
- entre 1 et 8 places par trajet, prix positif
- une seule réservation confirmée par passager et par trajet

Pourquoi une table des villes

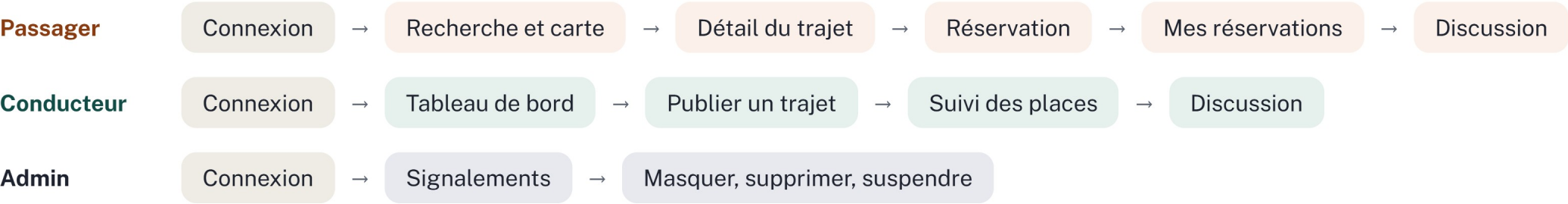
Une liste de villes avec leurs coordonnées, chargée au démarrage. Le conducteur choisit dans cette liste, donc la recherche retrouve toujours les trajets et la carte sait où placer les points, sans dépendre d'un service externe pendant la démo ni pendant les tests.

Un service de recherche d'adresses comme Nominatim peut venir plus tard.

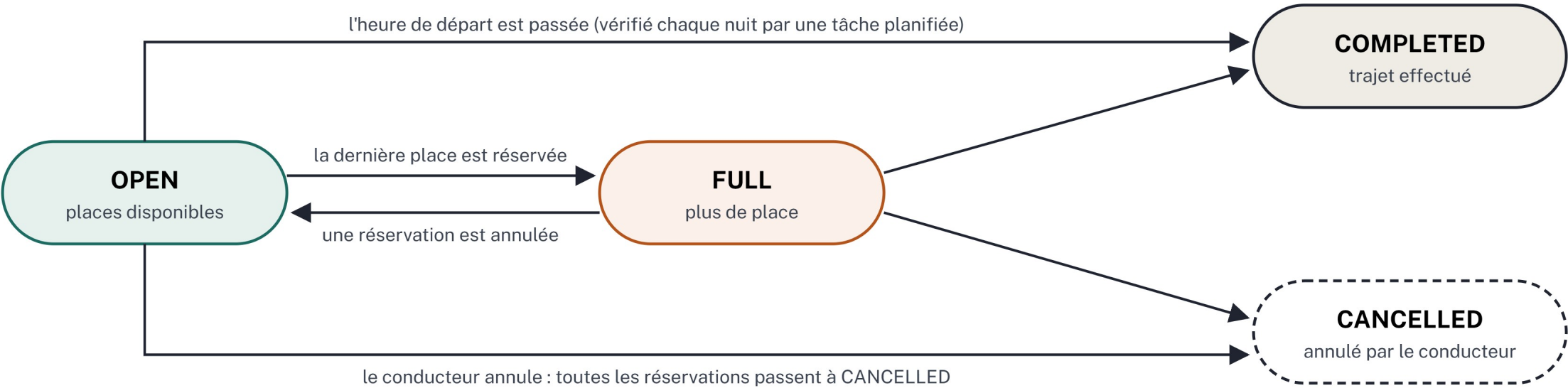
Parcours et états

Comment on circule entre les écrans, comment un trajet change de statut, et ce qui se passe quand deux passagers veulent la dernière place.

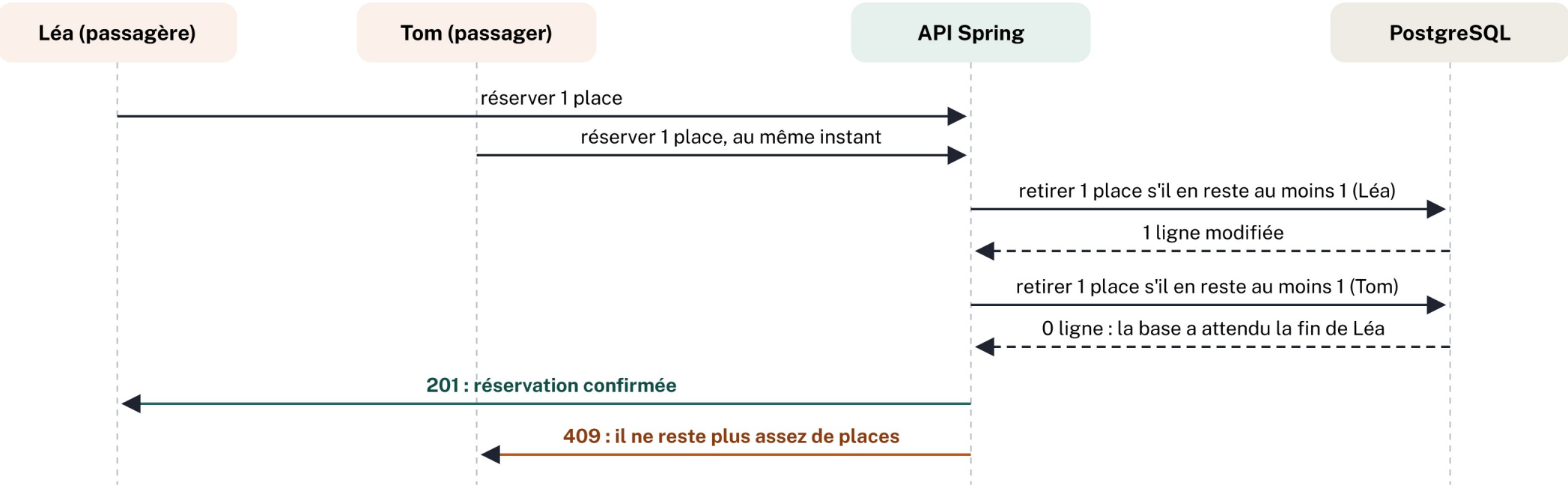
1. Les parcours



2. La vie d'un trajet



3. Il reste une place, deux passagers réservent en même temps



C'est exactement ce scénario que le test d'intégration rejoue : deux requêtes lancées en parallèle sur un vrai PostgreSQL (Testcontainers), puis on vérifie qu'une seule a réussi.